EXVUL

SMART CONTRACT AUDIT REPORT

SGY Smart Contract

OCTOBER 2026

Contents

1. EXECUTIVE SUMMARY **4**
1.1 Methodology 4

2. FINDINGS OVERVIEW **7**
2.1 Project Info And Contract Address 7
2.2 Summary 8
2.3 Key Findings 9

3. DETAILED DESCRIPTION OF FINDINGS **10**
3.1 Renouncing All Root Publication Roles Stops New Reward Authorization 10
3.2 First Halving Uses the Previous Monthly Budget as the Emission Base 12
3.3 Reward Algorithms and the 50/50 Allocation Lack Onchain Verification 14
3.4 Daily LP Proportional Burn Is Not Implemented 15
3.5 Missed Dates Are Not Accrued and Emission Phases Count Trigger Days 16
3.6 Expired Root Publication Window Requires Administrator Recovery 17

4. CONCLUSION **18**

5. APPENDIX **19**
5.1 Basic Coding Assessment 19
5.1.1 Apply Verification Control 19
5.1.2 Authorization Access Control 19
5.1.3 Forged Transfer Vulnerability 19
5.1.4 Transaction Rollback Attack 20
5.1.5 Transaction Block Stuffing Attack 20
5.1.6 Soft Fail Attack Assessment 20
5.1.7 Hard Fail Attack Assessment 21
5.1.8 Abnormal Memo Assessment 21
5.1.9 Abnormal Resource Consumption 21
5.1.10 Random Number Security 22
5.2 Advanced Code Scrutiny 22
5.2.1 Cryptography Security 22
5.2.2 Account Permission Control 22
5.2.3 Malicious Code Behavior 23
5.2.4 Sensitive Information Disclosure 23
5.2.5 System API 23

5.3 Project-Specific Context and Verification 24
Fees, Emissions, and Deployment Checks 24
Conditional Review Items 24
Validation and Deliverables 25

6. DISCLAIMER **28**

7. REFERENCES **29**

8. About Exvul Security **30**

1. EXECUTIVE SUMMARY

ExVul Web3 Security reviewed the supplied **SGY** smart contract implementation and mechanism description. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood**: represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact**: measures the technical loss and business damage of a successful attack.
- **Severity**: determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

Likelihood \ IMPACT	Informational	Low	Medium	High
High	INFO	MEDIUM	HIGH	CRITICAL
Medium	INFO	LOW	MEDIUM	HIGH
Low	INFO	LOW	LOW	MEDIUM

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs**: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing**: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations**: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
SGY	04/10/2026 - 04/10/2026	Solidity

Repository
https://github.com/sgy-finance/sgy.git

Commit Hash
`bfc392d5e2a8545c6d85a6a0c5ea6a635df1a143`

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	3
MEDIUM	3
LOW	0
INFO	0



2.3 Key Findings

Severity	Findings Title	Status
HIGH	Renouncing All Root Publication Roles Stops New Reward Authorization	Fixed
HIGH	First Halving Uses the Previous Monthly Budget as the Emission Base	Fixed
HIGH	Reward Algorithms and the 50/50 Allocation Lack Onchain Verification	Fixed
MEDIUM	Daily LP Proportional Burn Is Not Implemented	Fixed
MEDIUM	Missed Dates Are Not Accrued and Emission Phases Count Trigger Days	Fixed
MEDIUM	Expired Root Publication Window Requires Administrator Recovery	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 Renouncing All Root Publication Roles Stops New Reward Authorization

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`contracts/MinePool.sol`

DESCRIPTION:

Original issue M01: renouncing all reward-root publication roles stops authorization of new rewards.

Source locations: `contracts/MinePool.sol:40`, `contracts/MinePool.sol:109`

```
modifier onlyRootManager() {
    if (!hasRole(ROOT_MANAGER, msg.sender)) revert
   ErrorCallerNotRootManager();
    _;
}
function updateMerkleRoot(bytes32 merkleRoot_, uint256 merkleVersion_,
   uint256 newTotalReward_)
    external onlyRootManager
```

Subsequent rewards require a new Merkle root. The contract neither computes roots itself nor provides a permissionless process that verifies the complete reward calculation. If every `ROOT_MANAGER` and the final administrator renounce their roles, no new reward root can be published. Claims under an existing root may continue and the mining pool may still receive emissions, but no new reward allocation can be authorized. If root publication authority is retained, the publisher and its operational dependencies must be disclosed. If an automated contract performs this role, its source code and deployment evidence must be provided.

The TokenSGY `DEFAULT_ADMIN_ROLE` and `TOKEN_MANAGER`, the MinePool administrator and root publishers, and the Community owner are independent authorities. Renouncing one administrator role does not revoke the others. Fee-exempt identities also require separate disclosure.

IMPACT:

After all root publishers and the final administrator renounce their roles, the pool cannot authorize new rewards. Retaining a publisher preserves an operational dependency; claims already authorized by an existing root remain available.

RECOMMENDATIONS:

Before final role renunciation, replace the reward calculation and root-update process with a verifiable mechanism that can continue operating. Publish the complete role inventory, holder types, and role-change transactions.

3.2 First Halving Uses the Previous Monthly Budget as the Emission Base

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`contracts/TokenSGY.sol`

DESCRIPTION:

Original issue M02: the first halving uses the wrong emission base.

Source location: `contracts/TokenSGY.sol:234`; the preview function uses the same logic at `contracts/TokenSGY.sol:154`.

```
if (miningDay_ % (30*12*2) == 0) {
    rate_ = rate_ / 2;
    miningAddRate = rate_;
}
amount_ += amount_ * rate_ / _MAX_BASE;
miningAmount = amount_;
```

`amount_` is the previous month's emission budget. The specification requires the new phase to calculate 5% of cumulative issued supply, while the code increases the previous monthly budget by 5%. During the first 24 months at 10%, both recurrences coincide; they diverge at the first halving.

Month	Required Monthly Budget	Code Monthly Budget
1	2.10 million SGY	2.10 million SGY
2	2.31 million SGY	2.31 million SGY
24	18.8040 million SGY	18.8040 million SGY
25	**10.3422 million SGY**	**19.7442 million SGY**

Month 25 is approximately **90.91%** higher. At the end of month 24, cumulative issued supply is approximately 206.84 million SGY, so the specification applies 5% to that base. The code instead multiplies the previous monthly budget of approximately 18.8040 million SGY by 1.05, accelerating reserve release.

IMPACT:

After the first halving, the emission budget exceeds the stated cumulative-issued-supply model. Month 25 is approximately 90.91% higher, accelerating reserve release and causing both the preview and actual emission logic to diverge from the business rules.

RECOMMENDATIONS:

Track cumulative released supply, the current phase rate, and the monthly budget separately. Recalculate each month's budget from the specified base and update the preview function accordingly. Define whether transfer burns reduce this base; do not directly use `totalSupply()`, which includes unreleased reserves.

3.3 Reward Algorithms and the 50/50 Allocation Lack Onchain Verification

SEVERITY: HIGH **STATUS:** Fixed

PATH:

`contracts/MinePool.sol`

`contracts/Community.sol`

DESCRIPTION:

Original issue M03: the static and dynamic algorithms and the 50/50 allocation are not verified onchain.

Source locations: `contracts/MinePool.sol:66`, `contracts/Community.sol:27`

```
bytes32 leaf_ = keccak256(
    abi.encodePacked(account_, uid_, totalAmountStatic_,
    totalAmountDynamic_)
);
// After verifying membership under root_, pay the incremental static and
    dynamic cumulative rewards.
```

The proof verifies only that the publisher committed to the cumulative amounts. It does not prove that the amounts satisfy the reward algorithm. The current code does not implement descending ranking, a reasonable holding range, weighting, the 10,000 referral-line threshold, 10x amplification, or the cube root of the largest leg. It also does not verify that static and dynamic rewards each consume 50% of the global budget. Community records referral relationships only and does not connect them to these reward calculations in the reviewed source.

IMPACT:

The claim flow cannot guarantee the stated ranking, weighting, dynamic-computing rules, or the 50/50 global allocation. This finding is limited to an execution and verification gap in the reviewed source; whether an unavailable root generator follows the rules remains unverified.

RECOMMENDATIONS:

Provide the root generator, snapshot rules, and complete formulas, together with a reconciliation of aggregate allocations against emissions. If the system must operate autonomously under code-enforced rules, execute or verify these calculations onchain. Define the holding ranges, tie-breaking rules, computing units, and cube-root rounding method.

3.4 Daily LP Proportional Burn Is Not Implemented

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/TokenSGY.sol`

DESCRIPTION:

Original issue M04: the daily LP-proportional burn mechanism is absent.

Source location: `contracts/TokenSGY.sol:219`

```
uint256 balance_ = this.balanceOf(BLACK_HOLE);
uint256 amount_ = _getAndUpdateMiningAmount();
if (balance_ < amount_) amount_ = balance_;
super._transfer(BLACK_HOLE, MINEPOOL, amount_);
```

The mining path only transfers reserve tokens into MinePool. It does not read LP token holdings, pool reserves, or a pool ratio, and it performs no burn. Claims likewise only transfer tokens. The third advertised deflation mechanism is absent from the reviewed source.

IMPACT:

Daily emissions do not trigger a synchronized burn based on the pool ratio. Transfers between the reserve and MinePool do not reduce supply, so the third stated source of deflation is absent.

RECOMMENDATIONS:

Define the ratio numerator, denominator, observation time, and burn source. Then implement the calculation and supply reconciliation while preserving the 21 million SGY supply floor.

3.5 Missed Dates Are Not Accrued and Emission Phases Count Trigger Days

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/TokenSGY.sol`

DESCRIPTION:

Original issue M05: missed dates are not accrued and emission phases advance by trigger days.

Source locations: `contracts/TokenSGY.sol:219`, `contracts/TokenSGY.sol:248`

```
if (nowDay_ > lastDay) {
    lastDay = nowDay_;
    // Calculate and transfer only one daily budget.
}
miningDay += 1;
```

When multiple calendar days have elapsed, one trigger transfers only one day's budget and moves `lastDay` directly to the current day. Missed triggers delay release, unreleased tokens remain in reserve, and halving stages advance by successfully processed days. Therefore, 720 processed days are not equivalent to two calendar years.

IMPACT:

Missed dates do not accrue emissions, unreleased tokens remain in reserve, and phase transitions depend on successfully processed days. The implementation cannot guarantee halving every two calendar years.

RECOMMENDATIONS:

Define whether daily emissions may be skipped. If emissions accrue by calendar day, process the elapsed-day budget and phase boundaries while bounding catch-up work to avoid unbounded loops.

3.6 Expired Root Publication Window Requires Administrator Recovery

SEVERITY: MEDIUM **STATUS:** Fixed

PATH:

`contracts/MinePool.sol`

DESCRIPTION:

Original issue M06: an expired root-publication window requires administrator recovery.

Source locations: `contracts/MinePool.sol:109`, `contracts/MinePool.sol:126`

```
if (!(nextMerkleUpdateTime_ < block.timestamp
    && nextMerkleUpdateTime_ + 86400 > block.timestamp)) revert
   ErrorTimeWindow();
// Advance the date only after a successful publication.
merkleData.nextMerkleUpdateTime += 86400;
```

If a full publication window is missed, time can never satisfy the stale `nextMerkleUpdateTime` condition again. Only an administrator can call `setNextMerkleUpdateTime`. Therefore, even if a root publisher remains, one missed window can permanently block new roots after the final administrator renounces its role. Claims under existing roots remain available.

IMPACT:

After a full update window is missed, new root publication continues to fail. If the final administrator has renounced its role, the implementation cannot recover by resetting the schedule. Claims under existing roots can continue.

RECOMMENDATIONS:

Use a version or time-advancement rule that can cross missed dates, allowing an authorized publisher or a verifiable public update process to recover without a removed administrator.

4. CONCLUSION

The review of **SGY** identified **3 High and 3 Medium** mechanism and operational findings. All six are marked **FIXED** according to the remediation status provided by the project. Deployment, role state, reward producer and LP configuration should remain part of release verification.

Report production and integrity checks: **PASSED**. The six FIXED statuses reflect remediation status provided by the project. Production release verification should still confirm role configuration, the root-generation process, initial distribution, and LP configuration. This report does not warrant absolute absence of defects, market price performance, or investment returns.

To improve this report, we greatly appreciate constructive feedback or suggestions on our methodology, audit findings, or potential gaps in scope and coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

5.3 Project-Specific Context and Verification

This section records supplementary disclosures and conditional verification items from the original review. It does not add findings to Section 2.3. A "Not found"result in the generic assessment tables does not establish that deployment conditions not supplied for this review were verified.

Fees, Emissions, and Deployment Checks

- Default deductions on the selected pair total **3%** for buys and **5%** for sells, including a 1% burn in each direction. Before launch, an additional 30% produces default totals of 33% and 35%. The source sets the default launch time to **2026-10-06 20:08 Beijing time**. A transfer is fee-exempt when either endpoint is `INTERN_SYSTEM`. The mechanism description discloses only the burn rates; additional fees and exemptions should be disclosed, and live parameters require onchain verification. `contracts/TokenSGY.sol:36`
- The constructor mints 2.1 billion SGY at once: 21 million to `holder` and 2.079 billion to `BLACK_HOLE`. Daily emissions transfer this pre-minted reserve. The implementation can transfer tokens out of `BLACK_HOLE` through the internal mining path, so its full balance cannot be treated as permanently burned. `contracts/TokenSGY.sol:103`
- The constructor only creates the pair and transfers 21 million SGY to one holder. It does not implement the stated 16 million/5 million split or inject BNB liquidity. Distribution records, the BNB amount and valuation time, the LP-token recipient, and custody arrangements are needed to verify the USD 250,000 pool and USD 0.05 initial ratio. `contracts/TokenSGY.sol:95`
- Transfer burns stop when `totalSupply` reaches 21 million SGY. The code does not guarantee enough trading volume to reach that floor or any continuing increase in market price. `contracts/TokenSGY.sol:206`

Conditional Review Items

These items remain conditional or configuration risks. The supplied materials do not establish that they have caused unauthorized asset loss:

- **Initialization and cross-linking:** MinePool and Community initialization should be atomic with creation. Verify that `MinePool.SGY` and `TokenSGY.MINEPOOL` reference the same deployment. The constructor has no branch for an existing pair, so the live factory behavior must be checked.
- **Claim domain:** Proofs bind `uid`, while cumulative claimed amounts and per-version claim markers are stored by account. If different `uid` values are independent reward domains, their

amounts can interfere. If `uid` is only metadata for one account-level reward domain, that rule should be explicit. `contracts/MinePool.sol:17`

- **Referral queries:** The zero address can appear as an ancestor of an unregistered user. Traversal is capped at 100 levels, while ordinary registration has no equivalent depth limit. The root generator must use consistent rules for zero addresses, depth, and duplicate accounts. `contracts/Community.sol:39`
- **Fee conversion:** Conversion occurs before the final net amount reaches the pair. Compatibility should be verified with the live router, pair, liquidity operations, and lock configuration. Minimum output is fixed at 1 wei and lacks value-based output protection. The current evidence does not establish MEV or an automated-call-loop issue as a verified vulnerability. `contracts/TokenSGY.sol:199`
- **Privilege configuration:** Fee setters cap buy and sell rates at 100% without including the extra 30% pre-launch fee. A bad configuration can make total deductions exceed the transfer amount and revert. Before final role renunciation, verify the aggregate cap, calendar, thresholds, and recipient addresses that will become immutable. `contracts/TokenSGY.sol:114`

Validation and Deliverables

Byte-identical copies of the original contracts were tested in an isolated EVM: **12 tests across 2 suites, with 12 passed, 0 failed, and 0 skipped**. These assertions document current implementation behavior, including the mechanism differences above. Production source was not modified.

Issue	Validation Result
M01	After a valid publisher and the final administrator renounced their roles, new-root publication and schedule reset were rejected; a claim already authorized by the old root still succeeded once.
M02	After 720 daily mining calls, the first halving occurred on call 721. Preview and actual monthly budgets were both `19744236863777984058036008` wei; the expected value from the actual cumulative released base was `10342219309597991649447432` wei.

Issue	Validation Result
M03	Source review confirmed that algorithm and aggregate budget-ratio validation are absent. Full algorithm correctness awaits the root generator and exact definitions.
M04	With an SGY balance at the selected-pair placeholder, the next day's action increased MinePool by 70,000 SGY, reduced the reserve by 70,000 SGY, and left total supply unchanged.
M05	After the first day and a three-day gap, one trigger added only 70,000 SGY and increased the processed-day count by one. A repeated same-day trigger added nothing.
M06	Window boundaries, continued rejection after expiry, and inability to recover after final administrator removal were confirmed. The retained-administrator recovery control succeeded.

Additional checks confirmed that direct and repeated self-transfers did not increase balances, `transferFrom` self-transfers consumed allowance correctly, cumulative claims paid only the increment, and failed payments fully rolled back claim records.

Review materials: `contracts/analysis/solsec-20261004-150439/validation/test/MechanismValidation.t.sol`, `contracts/analysis/solsec-20261004-150439/validation/forge-test.stdout.log`, `contracts/analysis/solsec-20261004-150439/validation_results.json`, and `contracts/analysis/solsec-20261004-150439/VB.json`. Execution instructions are in `contracts/analysis/solsec-20261004-150439/validation/README.md`.

The build used native Solidity 0.8.19, 200 optimizer runs, and OpenZeppelin 4.9.2. The original project did not provide dependency locks or deployment configuration, and deployed bytecode was not matched. MinePool claim tests used a local standard ERC20; the complete live SGY-to-pool cross-deployment remains unverified. Constructor dependency stubs were used only for local instantiation, and the selected pair was a placeholder. **These tests do not verify live LP funds, reserves, price, or AMM callback compatibility.** Deployment evidence is still required for role renunciation, the root producer, initial distribution, and LP funding.

Review artifacts: `contracts/analysis/solsec-20261004-150439/project_understanding.json`, `contracts/analysis/solsec-20261004-150439/test_coverage_gaps.json`, `contracts/analysis/solsec-20261004-150439/all_functions.json`, `contracts/analysis/solsec-20261004-150439/func_risk.json`, `contracts/analysis/solsec-20261004-150439/pexp.json`, `contracts/analysis/solsec-20261004-150439/pexp_prioritized.json`, `contracts/analysis/solsec-20261004-150439/source_integrity.json`, `contracts/analysis/solsec-20261004-150439/pipeline_gates.json`, and `contracts/analysis/solsec-20261004-150439/emission_comparison.json`. The model includes long-term assumption-based calculations, while EVM emission tests reach the first halving boundary. The model does not represent the actual issuance timeline of a live deployment.

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement"or "disapproval"of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product"or "asset"created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

[1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound).
https://cwe.mitre.org/data/definitions/191.html.

[2] MITRE. CWE-197: Numeric Truncation Error.
https://cwe.mitre.org/data/definitions/197.html.

[3] MITRE. CWE-400: Uncontrolled Resource Consumption.
https://cwe.mitre.org/data/definitions/400.html.

[4] MITRE. CWE-440: Expected Behavior Violation.
https://cwe.mitre.org/data/definitions/440.html.

[5] MITRE. CWE-684: Protection Mechanism Failure.
https://cwe.mitre.org/data/definitions/693.html.

[6] MITRE. CWE CATEGORY: 7PK - Security Features.
https://cwe.mitre.org/data/definitions/254.html.

[7] MITRE. CWE CATEGORY: Behavioral Problems.
https://cwe.mitre.org/data/definitions/438.html.

[8] MITRE. CWE CATEGORY: Numeric Errors.
https://cwe.mitre.org/data/definitions/189.html.

[9] MITRE. CWE CATEGORY: Resource Management Errors.
https://cwe.mitre.org/data/definitions/399.html.

[10] OWASP. Risk Rating Methodology.
https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

EXVUL

Contact

Website

www.exvul.com

Email

contact@exvul.com

Twitter

@EXVULSEC

Github

github.com/EXVUL-Sec

ExVul